

Programmation Orientée Objet pour le Calcul Scientifique : Part I Survival kit

Organisation du cours

Sous forme d'**Item**, se raccrochant à trois catégories :

- Outils et Langage (Tool)
- Orienté objet (OO)
- **Gestion de la mémoire (Mem)**
- Programmation statique (Static)

Vous n'êtes pas seul : <https://en.cppreference.com/w/> est votre allié...

Item 9 : Vie et mort d'une variable en C++

En préambule : notion de **bloc de portée** ou **scope** 1/3

- un bloc de portée est défini entre **deux accolades**
- un bloc de portée permet de déclarer des **variables locales** :
 - toutes les variables déclarées dans le bloc sont **détruites en sortie de bloc**
 - le bloc définit donc une **durée de vie** pour des variables
- un bloc **hérite** de toutes les **variables précédemment définies**
- des blocs peuvent être **imbriqués** les uns dans les autres

Attention : un namespace n'est pas un bloc de portée

Item 9 : Vie et mort d'une variable en C++

En préambule : notion de **bloc de portée** ou **scope** 2/3

- Exemple de deux blocs imbriqués

```
int main() { // open main function scope
{
    int i = 0;
    {
        int j = i; // parent scope available
    }
    j = 2; // ERROR j undefined : out of scope
}
i = 0; // ERROR i undefined : out of scope
} // close main function scope
```

Item 9 : Vie et mort d'une variable en C++

En préambule : notion de **bloc de portée** ou **scope** 3/3

- Attention au *shadowing*

```
int main() {  
    {  
        auto i = 0;  
        {  
            auto i = 2; // first variable i unavailable  
        }  
        std::cout << i << "\n"; // i = 0 !  
        {  
            i += 2; // we refer to the first variable i  
        }  
        std::cout << i << "\n"; // i = 2 !  
    }  
}
```

Item 9 : Vie et mort d'une variable en C++

Sur la *pile* ou dans le *tas* ? 1/5

- On distingue trois zones mémoires différentes pour les variables
 - une zone '**statique**' (segments *data* ou *BSS* si variables non initialisées)
 - une zone '**automatique**' : la **pile** ou **stack**
 - une zone '**dynamique**' : le **tas** ou **heap**
- La zone mémoire où réside une variable est fonction de sa création
 - variable créé en dehors de tout bloc de portée => **statique**
 - variable créé dans un bloc de portée => **pile**
 - variable **alloué dynamiquement** => **tas**

Item 9 : Vie et mort d'une variable en C++

Sur la *pile* ou dans le *tas* ? 2/5

- variable créé en dehors de tout bloc de portée => statique

```
auto i = 3; // static zone (data segment)
int j; // static zone (BSS segment)

int main(){
    j = 10;
    std::cout << "i = " << i <<std::endl;
    std::cout << "j = " << j <<std::endl;
    return 0;
}
```

Item 9 : Vie et mort d'une variable en C++

Sur la *pile* ou dans le *tas* ? 3/5

- variable créé dans un bloc de portée => pile

```
void print() {  
    auto i = 1; // stack variable  
    std::cout << "i = " << i << std::endl;  
}  
  
int main(){  
    int j = 10; // stack variable  
    std::cout << "j = " << j << std::endl;  
    return 0;  
}
```

Item 9 : Vie et mort d'une variable en C++

Sur la *pile* ou dans le *tas* ? 4/5

- variable **alloué dynamiquement** => **tas**

Qu'est-ce qu'une allocation dynamique ?

- une réservation explicite d'un bloc mémoire de taille fixée
- utilisation des mots clés `malloc` (C) ou `new` (C++ legacy)
- l'intérêt sera de piloter la durée de vie de la variables
 - elle sera détruite manuellement par la commande `delete`
 - on manipule alors un **pointeur de variable** et non une variable : `*`

Item 9 : Vie et mort d'une variable en C++

Sur la *pile* ou dans le *tas* ? 5/5

- variable **alloué dynamiquement** => **tas**

Qu'est-ce qu'une allocation dynamique ?

```
int* int_ptr = new int{0}; // new returns an int address

*(int_ptr) = 5; // use * operator to get the pointer value

std::cout << "int_ptr = " << int_ptr; // pointer address (hexadecimal)
std::cout << "int value = " << *int_ptr; // pointed int value

delete int_ptr; // use delete to release memory
```

Exo 9.1 Créez et manipulez votre pointeur de réel (`float`).

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 1/6

- un pointeur est **l'adresse en mémoire d'une variable**
 - son type s'écrit `<type_pointé>*` ex `int* int_ptr; , float* float_ptr;`
 - on accède à la valeur pointée par l'opérateur `*` ex `*int_ptr = 1;`
 - on obtient l'adresse d'un objet par l'opérateur `&` ex `int* int_ptr = &int_var`
- une référence est **une vue sur une variable existante**
 - son type s'écrit `<type_référencé>&` ex `int& ref = existing_int_var;`
 - on la manipule comme la variable référencée ex `ref = 3;`
 - lorsque l'on **modifie** la **référence** d'une variable, on **modifie** la **variable**

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 2/6

```
int        i = 1;
int* int_ptr = &i; // int_ptr points on i
int& int_ref = i;  // int_ref references i

*int_ptr += 2; // i = 3
int_ref += 2;  // i = 5

std::cout << "        i = " << i;
std::cout << " int_ptr = " << int_ptr; // pointer address (hexadecimal)
std::cout << "*int_ptr = " << *int_ptr; // pointed variable value
std::cout << " int_ref = " << int_ref; // referenced variable value
```

Exo 9.2 Créer une variable réelle (float) et manipulez la via un pointeur et via une référence.

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 3/6

ces types pointeurs et références se combinent avec le mot clé `const`

```
int          i = 1;
int const*   const_int_ptr  = &i; // int_ptr points on i
int const&   const_int_ref  = i; // int_ref references i

*const_int_ptr += 2; // Error, points to a const int
const_int_ref += 2; // Error referenced a const int
```

Exo 9.3 Transformez vos pointeur/référence de l'Exo 9.2 pour qu'ils adressent un `const float`.

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 4/6

ces types pointeurs et références se combinent avec le mot clé `const` :

- attention à ne pas confondre pointeur constant et pointeur de constant

```
int i = 1;
int const* const_int_ptr = &i; // pointer of const
*const_int_ptr += 2; // ERROR, points to a const int

int j = 2;
const_int_ptr = &j; // Ok, the pointer is not const

int* const int_ptr_const = &i; // const pointer
int_ptr_const = &j; // ERROR pointer is constant
*int_ptr_const += 1; // ok, the pointed value is not const

int const * const const_int_const_ptr = &i; // Const pointer of const int !
```

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 5/6

Du coup quelle(s) différence(s) ?

- La principale différence est qu'un pointeur peut être nul et pas une référence, qui pointe forcément vers une variable ;

```
int* ptr = nullptr;  
int& ref; // ERROR
```

- un pointeur doit donc être manipulé avec précaution : déréférencer un pointeur null ou non initialisé = crash

```
int* ptr = nullptr;  
int i = *ptr; // COMPILES BUT CRASHES (segmentation fault)
```

Item 9 : Vie et mort d'une variable en C++

Pointeur et référence : tu références ou tu pointes ? 6/6

Du coup quelle(s) différence(s) ?

- La seconde différence est qu'un pointeur peut, dans sa durée de vie, pointer successivement vers plusieurs variables, une référence non.

```
int* ptr = nullptr;
int i = 10;
int j = 20;
ptr = &i;
ptr = &j;
int& ref = i; // will always points on i
ref = j; // i is modified (=j)
i++; // ref is modified
```

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la mémoire dans les signatures de fonction... 1/6

Deux comportements possibles pour passer un argument à une fonction :

- passage par **copie** appelé aussi par **valeur**;

```
void function( int i) {}
```

- passage par **référence**.

```
void function( int& i) {}
```

Pour les types de base du langage, **passage par référence** = argument en **entrée/sortie**

- **Exo 9.4** Créez une fonction qui modifie un argument passé en paramètre.

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la **mémoire** dans les **signatures de fonction**... 2/6

Deux comportements possibles pour passer un argument à une fonction :

- passage par **copie** appelé aussi par **valeur** :
dans la fonction, la variable est **une copie** de la variable passée à la fonction

```
void function(int i) {  
    i++; // local modification  
}  
int main() {  
    int i = 3;  
    function(i);  
    std::cout<< "i = " << i << std::endl; // i = 3 ! and not 4  
}
```

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la mémoire dans les signatures de fonction... 3/6

Deux comportements possibles pour passer un argument à une fonction :

- passage par référence :
dans la fonction, la variable est **une référence** de la variable passée à la fonction

```
void function(int& i) {  
    i++;  
}  
int main() {  
    int i = 3; // 'global' modification  
    function(i);  
    std::cout<< "i = " << i << std::endl; // i = 4  
}
```

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la mémoire dans les signatures de fonction... 4/6

Deux comportements possibles pour passer un argument à une fonction :

- passage par référence :
 - dans la fonction, la variable est une référence de la variable passée à la fonction
 - Attention on ne peut pas passer un temporaire par référence non constante

```
void function1(int& i) { i++;}  
void function2(int const& i) { std::cout <<i;}  
  
int main() {  
    function1(5); // ERROR  
    function2(5); // OK  
}
```

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la mémoire dans les signatures de fonction... 5/6

Passage d'un pointeur = un troisième comportement ?

- passage par pointeur

```
void function(int* i_ptr) {  
    if (i_ptr) (*i_ptr)++;  
}  
int main() {  
    int i = 3; // 'global' modification  
    function(&i);  
    std::cout<< "i = " << i << std::endl; // i = 4  
}
```

Nous l'appellerons le **passage par adresse** = l'argument est aussi en **entrée/sortie**

Item 9 : Vie et mort d'une variable en C++

Ne perdez pas la mémoire dans les signatures de fonction... 6/6

Passage d'un **pointeur** = un troisième comportement ? Pas vraiment...

- passage par **pointeur** = passage par **copie** d'une adresse ;

```
void function(int* i) {  
    std::cout << &i << "\n";  
}  
  
int main() {  
    int i = 10;  
    std::cout << &i << "\n"; // an address  
    function(&i); // another address  
}
```

La plupart du temps (sauf cas avec héritage) lui préférer un passage par référence.

Item 10 : Vie et mort d'un objet en C++

Comme les variables : toutes les règles vues à l'item 9 s'appliquent pour un objet

```
struct A{};
A a{}; // static variable

int main() {
    A a2{}; // stack variable
    A* a_ptr = new A{}; // heap variable
    delete a_ptr;
}
```

Focus sur trois points :

- **construction/desctruction** d'un objet,
- syntaxe avec les **pointeurs**,
- passage par **copie/référence** d'un objet

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 1/6

```
struct A{  
    int m_i = 0;  
}; // ce que vous voyez  
  
struct A{ // ce que vous ne voyez pas  
    A() {}; // constructor  
    ~A() {}; // destructor  
    A(A const& a) {m_i = a.i;} // copy operator  
    A& operator= (A const& a) {m_i = a.i;return *this;} // copy assignment  
    int m_i = 0;  
}; // move operators kept under silence for the moment
```

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 2/6

```
struct A{ // ce que vous ne voyez pas
    A() {}; // constructor
    ~A() {}; // destructor
    A(A const& a) {m_i = a.m_i;} // copy constructor
    A& operator= (A const& a) {m_i = a.m_i;return *this;} // copy assignment
    int m_i = 0;
}; // move operators kept under silence for the moment

int main(){
    A a; // A()
    A a2(a); // A (A const& a)
    A a3; // A()
    a3 = a2; // A& operator= (A const& a)
} //~A()
```

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 3/6

Le C++ génère automatiquement ces méthodes pour vous !

- Le constructeur : construit un objet vide ;
- le destructeur libère les ressources de l'objet,
- le constructeur par copie et l'opérateur d'affectation :
 - recopient les données membre à membre

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 4/6

Parfois il peut être nécessaire d'implémenter une de ces méthodes

- initialisation d'un membre (privé) dans le constructeur

```
class B {  
    int m_i;  
public:  
    B(int const i) : m_i(i) // member initialisation zone  
    {}  
}
```

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 5/6

Parfois il peut être nécessaire d'implémenter une de ces méthodes

- destruction d'une ressource particulière (ex pointeur)

```
class B {  
    int* m_ptr;  
public:  
    ~B() {delete m_ptr;}  
}
```

Item 10 : Vie et mort d'un objet en C++

Construction/destruction d'un objet : ce que vous ne voyez pas... 6/6

Parfois il peut être nécessaire d'implémenter une de ces méthodes

- initialisation d'un membre dans le constructeur
- destruction d'une ressource particulière (ex pointeur)

Plus aucune méthode n'est alors définie implicitement

- si une est explicitée ; il faut toutes les écrire...
- si on ne souhaite pas écrire de définition on peut utiliser `= default`
- pour supprimer une de ces méthodes on peut utiliser `= delete`

Exo 10.1 écrire une classe contenant explicitement les cinq méthodes implicites. Utilisez ces méthodes dans une fonction `main`. Bonus : empêchez la copie et l'affectation.

Item 10 : Vie et mort d'un objet en C++

Syntaxe avec les pointeurs

- La syntaxe change pour accéder aux données et aux méthodes d'un objet lorsque l'on le manipule via un pointeur :
 - on utilise l'opérateur `->` et non plus `.`

```
struct A {  
    int i = 0;  
    void print() const {std::cout << i << std::endl;};  
};  
int main(){  
    A* a_ptr = new A{};  
    a_ptr->i = 10;  
    a_ptr->print();  
    delete a_ptr;  
}
```

Item 10 : Vie et mort d'un objet en C++

Passage par copie/référence 1/3

Choix du type de **passage** d'un argument **objet** dans une **fonction** très important

- si on souhaite **modifier** l'objet, on passe par **référence** `&`
 - l'objet doit resté "alloué" tout le scope de la fonction (pas de temporaire !)
- si on ne souhaite **pas le modifier**, un passage par copie est souvent contre-indiqué
 - un objet peut être très lourd en mémoire (contenir un tableau)
 - la copie sera donc très coûteuse
 - on passe alors par **référence constante** `const&`
 - il n'y a alors pas de copie
 - l'objet est verrouillé en modification

Item 10 : Vie et mort d'un objet en C++

Passage par copie/référence 2/3

Choix du type de passage d'un argument objet dans une fonction très important

```
struct A {  
    int i[100000];  
};  
  
void modify_object (A& a) {  
    a.i[0] = 1;  
}  
  
void do_not_modify_object(A const& a) {  
    int val = a.i[0];  
}
```

Item 10 : Vie et mort d'un objet en C++

Passage par copie/référence 3/3

Choix du type de passage d'un argument objet dans une fonction très important

- si on **souhaite modifier** l'objet, on passe par **référence** `&`
- si on ne **souhaite pas le modifier**, on passe par **référence constante** `const&`

Exo 10.2 Construisez et appelez deux fonction, l'une prenant en argument une référence d'objet et l'autre une référence constante.

Si c'est **nécessaire** (souvent pour une programmation par interface) on peut passer

- un pointeur d'objet (paramètre en entrée/sortie) `void object_in_out(A*)`
- un pointeur d'objet constant (paramètre en entrée) `void object_in(A const*)`

Item 10 : Vie et mort d'un objet en C++

Exo 10.3 Créez une classe `A` ayant :

- un membre privé de type `int*` initialisé dans le constructeur.
- une méthode de modification (`set`)
- une méthode d'affichage (`print`) de l'entier contenu dans ce pointeur.

Dans un programme principal, construisez cet objet et passez-le dans deux fonctions :

- une fonction `print_object` qui appellera la méthode `print` de l'objet
- une fonction `modify_object` , qui prendra en paramètre un entier et appellera la méthode `set` de l'objet.

Attention aux fuites mémoires à la destruction de l'objet, ou lors de sa copie...